The Journal of Open Source Software

CD-Dynamax: A JAX-based Python package for continuous-discrete probabilistic state space modeling and inference

Matthew Levine [1,2,3¶], **Daniel Waxman** [2,3], **and Iñigo Urteaga** [4,5¶]

1 Broad Institute of MIT and Harvard, USA **2** Basis Research Institute, USA **3** Massachusetts Institute of Technology, USA **4** BCAM – Basque Center for Applied Mathematics, Spain **5** IKERBASQUE — Basque Foundation for Science, Spain ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

cd-dynamax is a JAX-based Python package for modeling, inference, and learning in ***continuous (time dynamics) - discrete (time observation)* state space models** (CD-SSMs). It provides a low-level, modular library of CD-SSM models and algorithms for systems where latent states evolve continuously in time according to stochastic differential equations and noisy observations are collected at discrete, possibly irregular, time instants.

cd-dynamax is designed to facilitate adoption, analysis, and experimentation of CD-SSMs by researchers and practitioners in various scientific domains. It provides (i) flexible CD-SSM model definitions of linear and nonlinear dynamics and observation models, (ii) state-of-the-art filtering, smoothing, and forecasting algorithm implementations, and (iii) system identification and model parameter estimation tools, all within a modern, efficient, and autodifferentiable JAX framework.

Mathematical background

Dynamical systems describe complex phenomena in fields including engineering, economics, neuroscience, ecology, and climate science. Continuous-discrete state space models (CD-SSMs) provide a probabilistic framework for systems where noisy observations are collected at irregular intervals while the latent process evolves as a stochastic differential equation (SDE) (Särkkä & Solin, 2019). A CD-SSM is described by:

- A (possibly unknown) stochastic dynamical system, i.e.,

$$dx(t) = f\left(x(t), u(t), t\right),\ dt + L\left(x(t), u(t), t\right),\ dw(t)\ ,$$

  where:
  - $x(t) \in \mathbb{R}^{d_x}$ and $x(0) \sim p(x_0; \varphi_{x_0})$,
  - $u(t) \in \mathbb{R}^{d_u}$ is an external input (i.e., control) signal,
  - $f : \mathbb{R}^{d_x} \times \mathbb{R}^{d_u} \times \mathbb{R} \to \mathbb{R}^{d_x}$ is the drift function,
  - $L : \mathbb{R}^{d_x} \times \mathbb{R}^{d_u} \times \mathbb{R} \to \mathbb{R}^{d_x \times d_w}$ is the diffusion coefficient, and
  - dw is the derivative of a d_w-dimensional Brownian motion with covariance Q.
- Data observed at arbitrary times $\{t_k\}_{k=1}^K$ via

$$p\left(y(t_k) \mid x(t_k), u(t_k), t_k; \varphi_y\right)\ .$$

  Gaussian subclasses recover $y(t_k) = h(x(t_k), u(t_k), t_k) + \eta_k$, where $h : \mathbb{R}^{d_x} \times \mathbb{R}^{d_u} \times \mathbb{R} \to \mathbb{R}^{d_y}$ and η_k are conditionally independent Gaussian noises.

The Journal of Open Source Software

We denote the collection of all parameters as $\theta = \{f,\ L,\ \varphi_{x_0},\ Q,\ \varphi_y\}$.

For discrete-time observations $Y_K = [y(t_1),\ \ldots,\ y(t_K)]$ and inputs $U_K = [u(t_1), \ldots, u(t_K)]$ of a CD-SSM, where inputs are assumed piecewise constant between observation instants, i.e., $u(t) = u(t_k)$ for $t \in [t_k, t_{k+1})$, the main inference and learning objectives are:

- **Filtering**: estimating the distribution of $x(t_K) \mid Y_K, U_K,\ \theta$.
- **Smoothing**: estimating the distribution of $\{x(t)\}_{t \le t_K} \mid Y_K, U_K,\ \theta$.
- **Forecasting**: estimating the distribution of $\{x(t)\}_{t > t_K} \mid Y_K, U_K,\ \theta$.
- **Parameter learning**: estimating $\theta \mid Y_K, U_K$, either point-wise or in distribution.

Statement of need

`cd-dynamax` is a JAX-based (Bradbury et al., 2018) open-source Python package that provides a **low-level library** for continuous-discrete state space modeling, inference, and learning.

Scientists modeling complex dynamical systems must capture continuous-time dynamics while accounting for the discrete and noisy nature of real-world observations. CD-SSMs provide a powerful framework for this, but the associated inference algorithms can be complex, numerically fragile, and computationally demanding, especially for nonlinear CD-SSMs (Murphy, 2023; Särkkä & Solin, 2019; Särkkä & Svensson, 2023).

Existing JAX-native SSM libraries, such as `dynamax` (S. W. Linderman et al., 2025) and `cuthbert` (Corenflos & Duffield, 2026), focus on discrete-time state space models and do not address continuous-time dynamics. `cd-dynamax` fills this gap by providing efficient implementations of state-of-the-art filtering, smoothing, forecasting, and parameter-learning algorithms for CD-SSMs within an autodifferentiable JAX framework that enables gradient-based system identification and modern Bayesian inference.

State of the field

Dynamical systems and state space models (SSMs) are core tools across many scientific disciplines. Python libraries for state space modeling (Aicher et al., 2025; Corenflos & Särkkä, 2021; Johnson, 2020; Lee et al., 2023; S. Linderman et al., 2020; Weiss et al., 2024) focus primarily on Hidden Markov Models, discrete-time SSMs, and corresponding Bayesian inference algorithms.

Amongst JAX-native libraries, `dynamax` (S. W. Linderman et al., 2025), `cuthbert` (Corenflos & Duffield, 2026), and `pfjax` (Lysy, 2023) are low-level codebases for discrete-time state space modeling and inference. `dynestyx` (Basis Research Institute, 2025) is a high-level library that integrates many of these packages, including `cd-dynamax`, for end-to-end Bayesian dynamical systems inference via NumPyro (Phan et al., 2019).

The `rodeo` (Wu & Lysy, 2025) library provides JAX-based probabilistic numerics tools for inference under deterministic dynamics (i.e., ordinary differential equations), but does not address stochastic dynamics.

To the best of our knowledge, no existing Python library provides a comprehensive, efficient framework for continuous-discrete state space modeling and inference.

Software design

`cd-dynamax` is driven by our vision of modularity, synergies with existing codebases, and extensibility for future research and applications.

We built `cd-dynamax` as a complement and extension to `dynamax` (S. W. Linderman et al., 2025), integrating `diffrax` (Kidger, 2021) for continuous-time dynamics. This lets us build on `dynamax`'s discrete-time SSM tools while supporting continuous-time dynamics, irregular sampling, and nonlinear and non-Gaussian models. By leveraging JAX-native differential equation solvers in `diffrax`, `cd-dynamax` supports hardware acceleration (GPU/TPU) and end-to-end automatic differentiation, enabling efficient gradient-based system identification and modern inference algorithms (e.g., Hamiltonian Monte Carlo).

A key design trade-off was balancing compatibility with the `dynamax` codebase and flexibility in nonlinear CD-SSM model specification and inference. We chose a decoupled architecture where CD-SSM model definitions are separated from inference implementations, such that practitioners can define a linear/nonlinear CD-SSM once and apply varied inference routines. This modular design allows (a) the integration of new model classes and inference algorithms; and (b) extension of the codebase to interact with probabilistic programming interfaces (e.g., NumPyro). `cd-dynamax` is designed to interact with any CD-SSM model (linear or nonlinear) in a unified way for model definition, state-inference, and system-identification, providing a consistently structured, scalable, and extensible framework for CD-SSMs.

We prioritized algorithmic reliability through a test suite that validates correctness against known benchmarks, including CD-SSMs with linear dynamics and discretized nonlinear CD-SSMs. This helps the library serve both methodological researchers extending inference theory and practitioners in systems biology, engineering, and finance who need robust handling of irregularly sampled data.

CD-dynamax modeling and inference

`cd-dynamax` offers:

1. A set of modular CD-SSM model definitions capturing linear and nonlinear dynamics and observation functions — with and without Gaussian assumptions — for irregular, noisy observation times.
2. Low-level, JAX implementations of probabilistic inference algorithms for filtering and smoothing in CD-SSMs (Särkkä & Solin, 2019; Särkkä & Svensson, 2023), including:
   - Kalman filtering and smoothing for linear Gaussian CD-SSMs.
   - Extended Kalman filtering and smoothing for nonlinear CD-SSMs.
   - Unscented and ensemble Kalman filtering for nonlinear CD-SSMs.
   - Differentiable particle filtering for nonlinear CD-SSMs.
3. A high-level interface for constructing and fitting probabilistic SSMs, with functions for:
   - point-estimation of model parameters via gradient-based or black-box optimization (e.g., `Scipy` (Virtanen et al., 2020), `Scipy-jaxopt` (Blondel et al., 2021)) of the (approximate) marginal log-likelihood.
   - Bayesian posterior parameter estimation, e.g., Markov Chain Monte Carlo (MCMC) via the BlackJAX (Cabezas et al., 2024) library.

The publicly available `cd-dynamax` documentation and examples describe CD-SSM modeling, filtering, smoothing, forecasting, and fitting for both experts and newcomers.

Research impact statement

`cd-dynamax`'s impact spans two complementary areas: (i) reproducible implementations that achieve state-of-the-art results on CD-SSM inference problems, and (ii) support for active and emerging collaborations in continuous-time dynamical systems modeling and inference.

Benchmarks and reproducible demos

The `cd-dynamax` repository includes tutorial notebooks and demo scripts demonstrating its ability to handle challenging CD-SSM problems with chaotic dynamics, partial observability, and noisy data:

- **CD-SSM Parameter uncertainty quantification**: fully Bayesian inference of the Lorenz-63 system's physical parameters, sampling posterior distributions (via MCMC) from noisy observations.
- **Learning chaotic neural SDEs from noisy observations**: learning unknown nonlinear dynamics from noisy, partially observed data using neural network parameterizations of the SDE drift, demonstrated on the Lorenz-63 system. We showcase accurate short-term forecasts and long-term statistical predictions.
- **Sparse identification of dynamical equations**: leveraging filtering-based likelihoods to perform Bayesian inference of library-coefficients from sparse, noisy trajectory data, demonstrating competitiveness with SINDy and related approaches (Brunton et al., 2016).

Collaborations and community development

`cd-dynamax` supports ongoing research collaborations and open-source development across multiple groups and use-cases, including:

- Applying CD-SSM inference to patient- and population-level phenotyping via physiologic models with researchers at University of Colorado, Dalhousie University, and Northwestern University. Related studies are in earlier planning stages on populations thought to follow continuous-time dynamics with unknown physiologic parameters.
- Applying CD-SSM inference to study drivers of collective animal behavior from video and acoustic tracking data with researchers at Basis Research Institute.
- Supporting graduate research at the Basque Center for Applied Mathematics (BCAM), including Master's theses on Bayesian modeling and system identification via `cd-dynamax`.
- `cd-dynamax` is a crucial back-end support for `dynestyx` (Basis Research Institute, 2025), a high-level library for end-to-end Bayesian inference for dynamical systems based on NumPyro (Phan et al., 2019). This validates `cd-dynamax`'s design as a modular, composable low-level library for larger probabilistic modeling ecosystems.
- Serving as an implementation substrate in developing new methods for dynamical systems inference (Waxman, 2025).

Several additional collaborations are planned, and we expect them to grow `cd-dynamax`'s user base and collaborative community.

AI usage disclosure

The core architectural design, feature development, and algorithmic implementations are the authors' original work, who carried out all scientific and technical decisions and judgments.

We acknowledge the use of generative AI to assist in the development and documentation of `cd-dynamax`:

- ChatGPT (OpenAI) assisted with code cleanup, plotting scripts, and drafts of docstrings and code comments.
- ChatGPT (OpenAI) and Gemini (Google) assisted with polishing the manuscript's narrative, flow, and clarity.

The codebase and manuscript were manually reviewed, tested, and edited by the authors. We verified the correctness of all AI-assisted code, and confirm that the final manuscript accurately reflects our research and design thinking.

The Journal of Open Source Software

Acknowledgements

Iñigo Urteaga acknowledges the support of ''la Caixa'' foundation's LCF/BQ/PI22/11910028 award and Grant RYC2023-045922-I by MICIU/AEI/10.13039/501100011033 and by ESF+, as well as funds by MICIU/AEI/10.13039/501100011033 and the BERC 2022-2025 program funded by the Basque Government. Matthew Levine acknowledges support by Basis Research Institute and the Eric and Wendy Schmidt Center at the Broad Institute of MIT and Harvard.

References

Aicher, C., Putcha, S., Nemeth, C., Fearnhead, P., & Fox, E. (2025). Stochastic gradient MCMC for nonlinear state space models. *Bayesian Analysis*, *20*(1). https://doi.org/10.1214/23-BA1395

Basis Research Institute. (2025). *Dynestyx: Bayesian inference for dynamical systems*. https://github.com/BasisResearch/dynestyx

Blondel, M., Berthet, Q., Cuturi, M., Frostig, R., Hoyer, S., Llinares-López, F., Pedregosa, F., & Vert, J.-P. (2021). Efficient and modular implicit differentiation. *arXiv Preprint arXiv:2105.15183*.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/google/jax

Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, *113*(15), 3932–3937. https://doi.org/10.1073/pnas.1517384113

Cabezas, A., Corenflos, A., Lao, J., & Louf, R. (2024). *BlackJAX: Composable Bayesian inference in JAX*. https://arxiv.org/abs/2402.10797

Corenflos, A., & Duffield, S. (2026). *Cuthbert: State-space model inference with JAX*. https://github.com/state-space-models/cuthbert

Corenflos, A., & Särkkä, S. (2021). *Code companion for Bayesian Filtering and Smoothing* (Version 1.0). https://github.com/EEA-sensors/Bayesian-Filtering-and-Smoothing

Johnson, M. J. (2020). *PyHSMM: Bayesian inference in HSMMs and HMMs* (Version 0.0.0). https://github.com/mattjj/pyhsmm

Kidger, P. (2021). *On Neural Differential Equations* [PhD thesis, University of Oxford]. https://docs.kidger.site/diffrax/

Lee, M. A., Yi, B., Martín-Martín, R., Savarese, S., & Bohg, J. (2023). *TorchFilter: Differentiable particle filtering in PyTorch*. https://github.com/stanford-iprl-lab/torchfilter

Linderman, S. W., Chang, P., Harper-Donnelly, G., Kara, A., Li, X., Duran-Martin, G., & Murphy, K. (2025). Dynamax: A python package for probabilistic state space modeling with JAX. *Journal of Open Source Software*, *10*(108), 7069. https://doi.org/10.21105/joss.07069

Linderman, S., Antin, B., Zoltowski, D., & Glaser, J. (2020). *SSM: Bayesian Learning and Inference for State Space Models* (Version 0.0.1). https://github.com/lindermanlab/ssm

Lysy, M. (2023). *PFJax: Particle filtering in JAX*. https://github.com/mlysy/pfjax

Murphy, K. P. (2023). *Probabilistic machine learning: Advanced topics*. MIT Press. http://probml.github.io/book2

Phan, D., Pradhan, N., & Jankowiak, M. (2019). Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv Preprint arXiv:1912.11554*.

Särkkä, S., & Solin, A. (2019). *Applied stochastic differential equations* (Vol. 10). Cambridge University Press.

Särkkä, S., & Svensson, L. (2023). *Bayesian filtering and smoothing* (Vol. 17). Cambridge University Press. https://doi.org/10.1017/CBO9781139344203

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Waxman, D. (2025). *Sequential Bayesian learning and ensembles for online inference and experimental design* [PhD thesis]. State University of New York at Stony Brook.

Weiss, R., Du, S., Grobler, J., Cournapeau, D., Pedregosa, F., Varoquaux, G., Mueller, A., Thirion, B., Nouri, D., Louppe, G., Vanderplas, J., Benediktsson, J., Buitinck, L., Korobov, M., McGibbon, R., Lattarini, S., Niculae, V., Gramfort, A., Lebedev, S., … Rockhill, A. (2024). *Hmmlearn* (Version 0.3.2). https://github.com/hmmlearn/hmmlearn

Wu, M., & Lysy, M. (2025). *Rodeo: Probabilistic methods of parameter inference for ordinary differential equations*. https://arxiv.org/abs/2506.21776